Assignment 2

Instructions

- Unless specified otherwise,
  - you can use anything in the Python Standard Library.
  - don't use any third-party library including NumPy in your code for this assignment.
  - you can assume that all user inputs and arguments to functions or methods are always valid and require no validation or exception handling.
- The blue text in the Sample Run(s) section of each question refers to the user inputs. The symbol ↵ means the Enter/Return key.
- Please follow closely the format of the sample output for each question. Your program should produce **exactly** the same output as the sample (same text, letter case, punctuation, spacing, etc.). The output for each question should end with a single newline character, which has been provided by the `print()` function by default.
- Please follow exactly the specified name, parameter list and return value(s) for the functions to be written if a question has stated (to facilitate our grading). You may define additional (inner) functions for specific tasks or further task decomposition in any question if you see fit.
- **For questions that require you to define specific functions, your main client code (for calling and testing the functions) must be put under an `if __name__ == '__main__'`: check because we may import your script into our test scripts calling your functions in our own ways for grading.**
- **Name your script files as `q1.py` for Question 1, `q2.py` for Question 2, … (case-sensitive). Using other names affects our marking and may result in deduction.**
- You are highly encouraged to annotate your functions to provide type information for parameters and return values, and to include suitable comments in your code to improve the readability. Feel free to modify or extend any comments provided in our starter code if you see fit.
- You should follow the standard PEP8 style guide for writing Python code. Install the `pycodestyle` package via `pip install pycodestyle`. Run the command `pycodestyle qn.py` (*n* = 1, 2, …) to check for any PEP8 style violations, which may lead to minor mark deductions. You should fix all the detected issues before submission.

Question 1 (30%)

In this question, we are going to analyze a dataset about car parks in Hong Kong using a clustering algorithm. Solving this question requires integrated use of programming skills including manipulation of dictionaries, file I/O, JSON data processing, exception handling, and basic object-oriented programming.

Write a program `q1.py` that defines several functions and classes specified as follows. Download the starter code file `q1_starter.py`, rename it, and add your implementation.

(a) Data Retrieval

Function Signature:
```
get_json_data(no_cache: bool = False)
                -> list[dict[str, Any]]
```

This function implements the logic to download the car park data from the Data API provided by the Transport Department of the Hong Kong SAR government.

Data Source: The dataset is provided in JSON format at:
https://resource.data.one.gov.hk/td/carpark/basic_info_all.json

The JSON structure contains a top-level dictionary with a key:

```
{
    "car_park": [ {...}, {...}, ... ]
}
```

Each item in the `"car_park"` list is a dictionary that describes one car park such as its park ID, name, address, and location specified by latitude and longitude.

Caching: To avoid unnecessary download overhead, this function will first check if there is a cached copy of the data saved as a local file in the current directory. This file is called `"carparks.json."` The parameter `no_cache`, when set to `True`, forces the function to bypass loading the cached file and to always download fresh data from the specified URL.

Function Logic:

1. If `no_cache` is `False` and if the cache is available (i.e., `carparks.json` exists), print the message "*Retrieving data from cached file ...*", load the file `carparks.json`, and return the parsed car park data as a list of dictionaries.
2. Otherwise, i.e., if `no_cache` is `True` or the cache is unavailable (i.e., opening `carparks.json` raises a `FileNotFoundError`), print the message "*Retrieving data from the web ...*", download the JSON data from the URL, save it to `carparks.json` for future use, and return the parsed data as a list of dictionaries.

Notes:

1. You may use the `requests` package to simplify the JSON download operation.
2. The JSON response returned by this URL contains a UTF-8 Byte Order Mark (BOM), which is a specific byte sequence (EF BB BF) placed at the beginning of a text stream to indicate that the content is encoded in UTF-8 (for example, when Chinese characters are present).
3. If you use the built-in `Response.json()` method provided by the `requests` package to parse this JSON, it may raise a `requests.exceptions.JSONDecodeError` if it encounters this unexpected BOM. To avoid this issue, you should manually decode the response content using the `utf-8-sig` codec, which looks for the presence of a BOM and if found, treats it as metadata and effectively strips it from the content. Typical code is as follows:

   ```
   response = requests.get(URL)
   ...
   json_string = response.content.decode('utf-8-sig')
   ```

4. You can then call the standard `json.loads(…)` function to parse the JSON string.
5. When dumping the JSON data to the cache file (`carparks.json`), use an indentation of 4 spaces and set the `ensure_ascii` argument to `False` (for storing Chinese characters in the JSON file properly instead of storing the Unicode numbers, e.g., "天" instead of "\u5929").
6. Suppose that `response` is a `Response` object. Calling `response.raise_for_status()` in the `try` block will raise an `HTTPError` if a bad HTTP status (4xx, 5xx) occurs.
7. Handle exceptions that may occur during HTTP requests or JSON parsing gracefully. Specifically, only the types of errors in the following table need to be handled; all others can be ignored. Let e denote the exception object being caught.

Type of e	**f-string to be printed**
`requests.exceptions.HTTPError`	`"HTTP error: {e}"`
`requests.exceptions.RequestException`	`"Ambiguous exception: {e}"`
`json.JSONDecodeError`	`"JSON decoding error: {e}"`

(b) The `Carpark` Class

The dictionary (JSON object) of a car park contains more information than necessary for our data analysis. For abstraction sake, we select only five data fields (`park_id`, `name_en`, `district_en`, `latitude`, and `longitude`) to represent a car park. Create a class named `Carpark` to represent a single car park. This class has the following attributes and methods.

Attributes:

- **`park_id`**: stores the `park_id` property in the car park's JSON
- **`name`**: stores the `name_en` property in the car park's JSON
- **`district`**: stores the `district_en` property in the car park's JSON
- **`latitude`**: stores the `latitude` property in the car park's JSON
- **`longitude`**: stores the `longitude` property in the car park's JSON

Methods:

- **`__init__(self, data: dict)`**: Accept a dictionary representing a car park (i.e., one item from the JSON list) and initialize the attributes of the object using the dictionary's entries.
- **`__repr__(self)`**: Return the `park_id` string.
- **`__str__(self)`**: Return a string in the format: "`name (park_id)`".

(c) Group by District

Function Signature: `group_by_district(json_data: list[dict[str, Any]]) -> dict[str, list[Carpark]]`

This function reorganizes the list of car park dictionaries into a dictionary where the keys are the English district names (`district_en`) and the values are lists of `Carpark` objects corresponding to those districts.

The expected structure of the resultant dictionary (the return value) is as follows:

```
{
    ...
    "Wong Tai Sin": [<Carpark object>, <Carpark object>, ...],
    "Kwai Tsing": [<Carpark object>, <Carpark object>, ...],
    ...
}
```

(d) Summary Printing

Function Signature: `print_summary(data: dict[str, list[Carpark]]) -> None`

This function prints a summary table showing car park statistics for each district. The table includes the following columns:

- The district name.
- The midpoint (average latitude and longitude) of all the car parks within that district, i.e.,

$$midpoint_A = \left(\frac{\sum_{i=1}^{N} latitude_i}{N}, \frac{\sum_{i=1}^{N} longitude_i}{N}\right) \text{ where } N = \#\text{car parks in district } A$$

- The total number of car parks in the district.

Format the output in a tabular structure as shown below.

```
         1         2         3         4         5
12345678901234567890123456789012345678901234567890123456789012345678
District             Midpoint of car parks     # Car parks
-------------------------------------------------------------
Southern             ( 22.24655, 114.16571)    18
Eastern              ( 22.27442, 114.23161)    31
Wan Chai             ( 22.27517, 114.18281)    12
Central & Western    ( 22.28293, 114.14883)    10
```

Note: The top two lines of blue numbers are included only to illustrate the spacing requirements; they are not part of the actual output. For each value in the midpoint tuple, use a width field of 9 characters and display 5 decimal places.

(e) The KMeans Class

Background:

K-means clustering is a popular *unsupervised* machine learning algorithm used for partitioning a set of data points into k *clusters* such that points in the same cluster are more similar to each other than to those in other clusters. Each cluster is represented by its *centroid*, which is the mean of all points assigned to that cluster. The steps of the clustering algorithm to be implemented in this part are outlined as follows

1. Initialization:

- Suppose that the number of clusters to form from the dataset is k. Randomly select k data points in the dataset as initial centroids. Let's call this list of length k the `centroids` list.

2. Assignment Step:

- For each data point, calculate the Euclidean distance from the point to each centroid in `centroids`.
- Assign each data point to the nearest centroid to form the clusters. Suppose that each cluster has a cluster ID (or label) and a list of data points that are assigned to it. If `clusters` is a list of clusters whose indexes equal their IDs, then `clusters[c].append(p)` will append the point p to the cluster with ID c.

3. Update Step:

- For each empty cluster with no data points assigned, set its centroid to (0, 0).
- For each non-empty cluster, compute the new centroid as the mean of all data points assigned to that cluster. That is, given a cluster j,

$$C_j = \left(\frac{\sum_{i=1}^{N_j} x_i}{N_j}, \frac{\sum_{i=1}^{N_j} y_i}{N_j} \right)$$

  where
  - C_j is the updated centroid for cluster j.
  - N_j is the number of data points assigned to cluster j.
  - x_i and y_i are the latitude and longitude of the i-th data point belonged to cluster j.
- Update the `centroids` list with the new centroids computed.

4. **Convergence Check:**

- Check if the centroids have changed. If all centroid shifts (i.e., every Euclidean distance between each pair of new and old centroids) are less than a predefined tolerance, the algorithm has converged and can stop.
- Otherwise, go back to Step 2 to reassign the data points.

5. **Output:**

- The outputs of clustering are the final centroids and cluster assignment labels.

Implementation:

Implement the `KMeans` class to perform the k-means clustering algorithm described above on the geographical coordinates of the car park dataset. You must implement it from scratch without using any third-party packages such as Scikit-learn.

To facilitate both your implementation and our grading (via unit testing), you are required to follow the class skeleton provided in the starter code. In particular, the given method signatures most not be modified. Some methods whose names begin with an underscore are private methods intended for internal use within the class. Note that our OO design (including attribute and method names) are a simplified version that closely resembles the `KMeans` class in the Scikit-learn package.

Attributes:

- **`n_clusters`** (`int`): the predefined number of clusters, i.e., k in the k-means algorithm.
- **`max_iter`** (`int`): the maximum number of iterations for repeating steps 2 to 4.
- **`tot`** (`float`): the predefined tolerance threshold for convergence check.
- **`centroids`** (`list[Point]`): the list of coordinates of the cluster centers. Note: `Point` is a type alias defined in the starter code to represent a pair of coordinates.

Methods:

- **`__init__(self, n_clusters=3, max_iter=100, tol=1e-5)`**: Initialize the attributes of the `KMeans` object with the corresponding arguments and set `centroids` to an empty list. Note: The client code will pass 18 for `n_clusters` for the 18 districts in Hong Kong.

- **`fit(self, data: list[Point]) -> None`**: This method implements the main loop of the k-means clustering algorithm presented above and recapped briefly as follows.

  Step 1: use `random.sample()` to randomly select `n_clusters` initial centroids from `data`.

  Then repeat up to `max_iter` times:
  Step 2 (**Assignment Step**): Assign each point in `data` to its nearest centroid.
  Step 3 (**Update Step**): Compute new centroids as mean of cluster points.
  Step 4 (**Convergence Check**): Stop if centroid movement < `tol` for all centroids.

  Your implementation of `fit()` must use the following private methods to allow us to perform unit testing.

- **`_closest_centroid(self, point: Point) -> int`**: Return the index (or cluster label) of the centroid closest to the specified `point`. To determine this index, you may create a temporary list of Euclidean distances from each centroid to the given point and then find the index for the smallest distance in that list.

- **`_euclidean_distance(self, point1: Point, point2: Point) -> float`**: Compute the Euclidean distance between `point1` and `point2`. Mathematically, given two points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$, this distance is given by:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **`_mean(self, cluster: list[Point]) -> Point`**: Compute the midpoint of all points in the specified `cluster` list. Note: If the cluster is empty, simply return `[0, 0]`.

- **`_has_converged(self, old_centroids: list[Point], new_centroids: list[Point]) -> bool`**: Return `True` if all centroid shifts (i.e., every Euclidean distance between each pair of new and old centroids) are less than `tol`, and `False` otherwise.

- **`predict(self, data: list[Point]) -> list[int]`**: Return the list of cluster labels for all the points in `data`. This list has the same length as `len(data)`. Hint: simply reuse the `_closest_centroid()` method to generate each item in this list. Each item is the index to the closest centroid for each point in `data`.

Sample Client Code:

```python
if __name__ == "__main__":
    # Retrieve the raw data about car parks
    json_data = get_json_data()

    # Group the data and print a summary
    grouped_data = group_by_district(json_data)
    print_summary(grouped_data)

    # Print some example car parks
    print("Example Car Parks:\n" + "-" * 18)
    for district in ['Eastern', 'Islands', 'Sha Tin']:
        print(f"First 5 car parks in {district}:", grouped_data[district][:5])
        for i, carpark in enumerate(grouped_data[district][:5]):
            print(i + 1, carpark)

    # Prepare the data points for clustering
    data_points = []
    for carpark in json_data:
        cp = Carpark(carpark)
        data_points.append((cp.latitude, cp.longitude))

    # Run the k-means clustering algorithm
    random.seed(42)  # Fix the seed for deterministic result
    kmeans = KMeans(n_clusters=18)
    kmeans.fit(data_points)

    # Print the clustering result
    print("Clustering Result:\n" + "-" * 18)
    print("Label     Centroid")
    for i, (latitude, longitude) in enumerate(sorted(kmeans.centroids)):
        print(f"{i:<10d}({latitude:9.5f}, {longitude:9.5f})")

    labels = kmeans.predict(data_points[:10])
    print("Predicted clusters:", labels)  # Print labels for the 1st 10 points
```

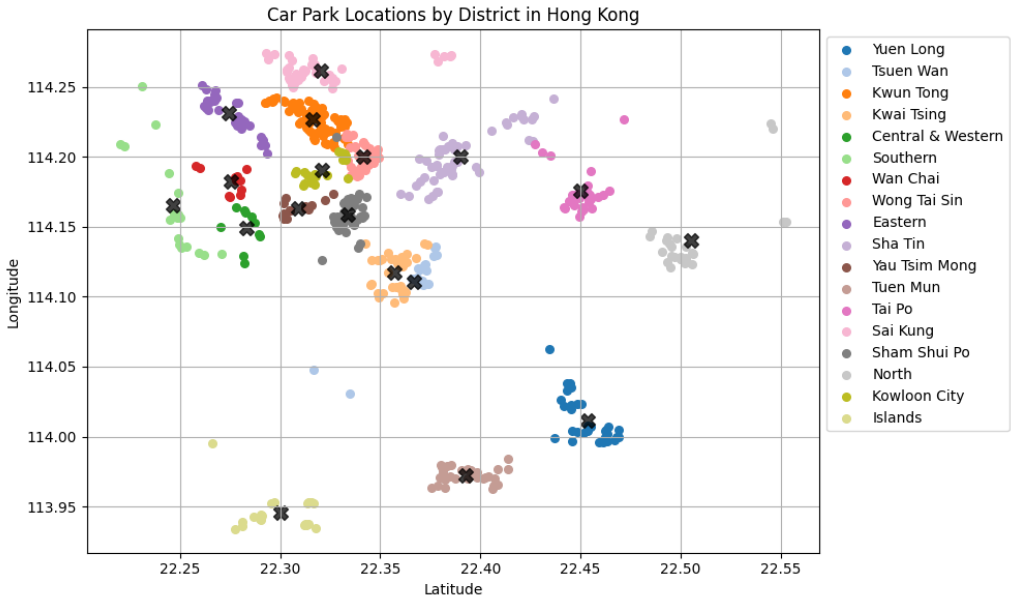
Sample Run:

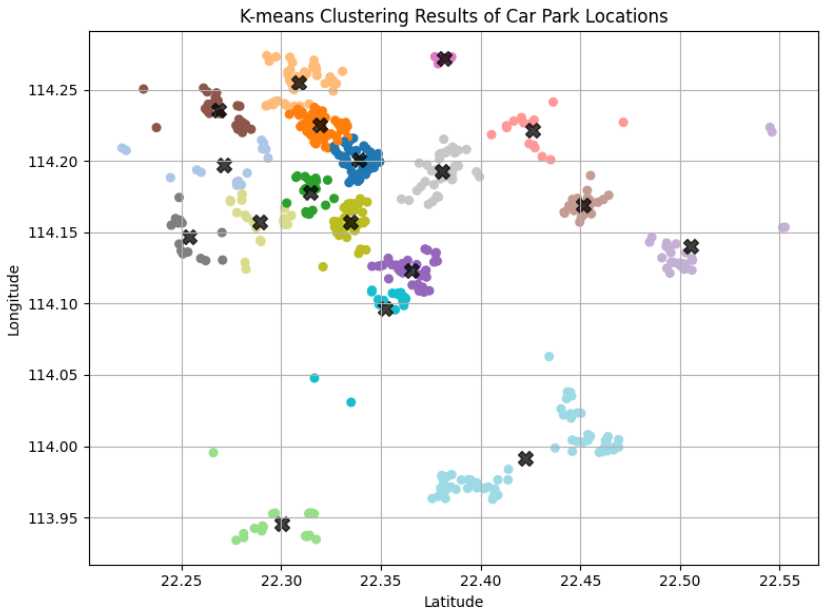
```
Retrieving data from the web ...
District             Midpoint of car parks     # Car parks
-------------------------------------------------------
Southern             ( 22.24655, 114.16571)    18
Eastern              ( 22.27442, 114.23161)    31
Wan Chai             ( 22.27517, 114.18281)    12
Central & Western    ( 22.28293, 114.14883)    10
Islands              ( 22.30017, 113.94560)    22
Yau Tsim Mong        ( 22.30894, 114.16345)    14
Kwun Tong            ( 22.31633, 114.22683)    70
Sai Kung             ( 22.32030, 114.26205)    40
Kowloon City         ( 22.32098, 114.19054)    18
Sham Shui Po         ( 22.33400, 114.15912)    37
Wong Tai Sin         ( 22.34125, 114.20023)    38
Kwai Tsing           ( 22.35682, 114.11701)    33
Tsuen Wan            ( 22.36706, 114.11040)    18
Sha Tin              ( 22.39005, 114.20052)    51
Tuen Mun             ( 22.39279, 113.97243)    33
Tai Po               ( 22.45013, 114.17552)    27
Yuen Long            ( 22.45340, 114.01162)    32
North                ( 22.50530, 114.14010)    27
Example Car Parks:
------------------
First 5 car parks in Eastern: [tdcp9, tdc14p1015, tdc14p1016, tdc14p1017,
tdc14p1030]
1 Shau Kei Wan Car Park (tdcp9)
2 Oi Tung Car Park (tdc14p1015)
3 Yiu Tung Car Park 1 (tdc14p1016)
4 Hing Tung Car Park (tdc14p1017)
5 Siu Sai Wan Phase 3 Car Park (tdc14p1030)
First 5 car parks in Islands: [tdc14p6728, tdc14p6729, tdc14p6730, tdc2p8, tdc2p1]
1 Yat Tung Car Park 2 & 3 (Sin Yat) (tdc14p6728)
2 Yat Tung Car Park 1 (tdc14p6729)
3 Fu Tung Car Park A (tdc14p6730)
4 Airport Car Park - Urban & Lantau Taxi (HOURLY) (tdc2p8)
5 AIRPORT CAR PARK 1 (HOURLY) (tdc2p1)
First 5 car parks in Sha Tin: [tdstt2p2, tdstt38, tdc14p2101, tdc14p2104,
tdc14p2106]
1 Value Parking Shan Mei Street (tdstt2p2)
2 Man Lai Road (tdstt38)
3 Chung On Car Park (tdc14p2101)
4 Heng On Car Park (tdc14p2104)
5 Yiu On Car Park (tdc14p2106)
Clustering Result:
------------------
Label     Centroid
0         ( 22.25380, 114.14666)
1         ( 22.26849, 114.23600)
2         ( 22.27106, 114.19697)
3         ( 22.28938, 114.15755)
4         ( 22.30017, 113.94560)
5         ( 22.30854, 114.25497)
6         ( 22.31452, 114.17815)
7         ( 22.31927, 114.22512)
8         ( 22.33439, 114.15705)
9         ( 22.33868, 114.20093)
10        ( 22.35226, 114.09635)
```

```
11        ( 22.36518, 114.12342)
12        ( 22.38057, 114.19293)
13        ( 22.38163, 114.27183)
14        ( 22.42263, 113.99172)
15        ( 22.42595, 114.22143)
16        ( 22.45168, 114.16949)
17        ( 22.50530, 114.14010)
Predicted clusters: [17, 7, 3, 7, 7, 15, 12, 1, 0, 15]
```

Note: We will NOT grade your client code but the functions only in this question. You may modify the client code for testing more test cases, generating more output that differs from the above.

Scatterplots: We also provided an extra script called `q1_plot.py`, which can make two scatterplots to compare the car park data with true labels (districts) and the clustering results (unknown labels). If your `q1.py` is implemented correctly, you should be able to run `q1_plot.py` to create similar plots. This is an optional step that you could try to double check if your implemented clustering works as expected. This script requires the [Matplotlib] package; it can be installed via the command `pip matplotlib`.





In the first figure, the points (car parks) in the same district have the same color. Each black cross marks the midpoint of all the car parks in the same district.

In the second figure, the points in the same cluster have the same color. The black crosses denote the centroids of the clusters.

Note: The exact color used per district or cluster does not matter.

Question 2 (20%)

In this question, you are going to build a simple text classification system using the *Naive Bayes* algorithm. The goal is to create a classifier that can categorize text documents into different classes based on their content. For simplicity, a text document in our context is just one sentence long without any punctuation (except single quote) or numbers. Before programming, let's learn the theory behind in case you have no ideas. Basic understanding of *conditional probability*, i.e., the probability of an event occurring given that another event has already occurred, is required.

Background:

1. Bayes' Theorem

Naive Bayes classifiers are based on Bayes' Theorem, which provides a way to calculate the probability of a class given a document. The core formula is:

$$P(c|d) = \frac{P(d|c) \cdot P(c)}{P(d)} \tag{1}$$

where

- $P(c|d)$ is the posterior probability of class c given document d.
- $P(d|c)$ is the likelihood of document d given class c.
- $P(c)$ is the prior probability of class c.
- $P(d)$ is the total probability of document d across all classes.

2. Class Prior Probability

The prior probability $P(c)$ is calculated as:

$$P(c) = \frac{N(c)}{N} \tag{2}$$

where

- $N(c)$ is the count of documents in class c.
- N is the total number of documents.

3. Likelihood Calculation

The likelihood of a document d given a class c, $P(d|c)$, is computed as the <u>product</u> of the probabilities of each word in the document d given the class c:

$$P(d|c) = P(w_1, w_2, \dots, w_n|c) = P(w_1|c) \cdot P(w_2|c) \cdots P(w_n|c) = \prod_{i=1}^{n} P(w_i|c) \tag{3}$$

where w_1, w_2, …, w_n are the words in the document d.

4. Word Probability Calculation with Laplace Smoothing

However, from this equation, when a word, say w_0, in a document does not appear in the training data for a specific class, then $P(w_0|c) = 0 \Longrightarrow P(d|c) = 0 \Longrightarrow P(c|d) = 0$, hence the likelihood of that class given the document becomes zero. Thus, the classifier will not predict that class, potentially ignoring relevant classes. This is referred to as the *problem of zero probability*.

To handle this problem in Naïve Bayes, a smoothing technique called *Laplace smoothing* is commonly employed. For each word w given class c, the probability calculated using *Laplace smoothing* is:

$$P(w|c) = \frac{N(w,c) + \alpha}{N(c) + \alpha \cdot K} = \frac{N(w,c) + 1}{N(c) + |V|} \tag{4}$$

where

- $N(w, c)$ is the count of how many times word w appears in documents of class c.
- $N(c)$ is the total count of all words in documents of class c.
- V is the *vocabulary*, i.e., set of all unique words; $|V|$ is the total number of unique words.
- K is the number of dimensions (or features) in the data. In this application, the number of features is equal to the size of the vocabulary, so $K = |V|$.
- α is the smoothing parameter ($\alpha > 0$), we set α to 1.

5. Classification

To classify a new document, the classifier calculates the posterior probability for each class and selects the class with the highest probability. So, the predicted class, denoted by $\hat{c}$, is given by

$$\hat{c} = \underset{c}{\operatorname{argmax}}\, P(c|d) = \underset{c}{\operatorname{argmax}}\, \frac{P(d|c) \cdot P(c)}{P(d)} \tag{5}$$

Since $P(d)$ is the same for all classes, it can be ignored during the classification step for simplicity:

$$\hat{c} = \underset{c}{\operatorname{argmax}}\; P(d|c) \cdot P(c) \tag{6}$$

Note: the argmax operator means selecting an argument such as the expression attains the maximum. Here, it means selecting a class c such that the probability, $P(d|c) \cdot P(c)$, is the highest.

Substituting (3) into (6), we have

$$\hat{c} = \underset{c}{\operatorname{argmax}}\; P(c) \cdot \prod_{i=1}^{n} P(w_i|c) \tag{7}$$

Like calculations for language modeling, Naive Bayes calculations are usually done in log space to avoid underflow and increase speed. The formula becomes:

$$\hat{c} = \underset{c}{\operatorname{argmax}}\; \log P(c) + \sum_{i=1}^{n} \log P(w_i|c) \tag{8}$$

This computes the predicted class as a linear function of input features. Thus, Naive Bayes is called a *linear classifier*.

In summary, the Naive Bayes classifier calculates the posterior probability of each class based on the prior probabilities and the likelihoods of the words in the document, applying Laplace smoothing to handle zero probabilities (in other words, we need to compute using (4) and (8) together). The class with the highest posterior probability is chosen as the predicted class for the document.

Implementation:

Write a program `q2.py` that defines a class `NaiveBayesClassifier` specified as follows. Download the starter code file `q2_starter.py`, rename it, and add your implementation.

The `NaiveBayesClassifier` class has the following instance variables and methods.

Instance Attributes:

- **`class_priors`** (type: `dict[str, float]`): a dictionary storing all the class labels as keys and their prior probabilities as values.

- **`word_counts`** (type: `dict[str, dict[str, int]]`): a nested dictionary storing class labels as keys and per-class word count maps as values. For a class label c, its word count map is a dictionary with each key denotes a word w in documents of class c; the value of each key is the count of how many times word w appears in documents of class c.

- **`vocabulary`** (type: `set[str]`): the set of unique words that have ever appeared in all documents.

Instance Methods:

(a) Initialization:

- `__init__(self)`: the constructor initializes all the instance variables to empty data structures.

(b) Training:

- `fit(self, X: list[str], y: list[str]) -> None`: this method trains the classifier by iterating through a list of documents X and their corresponding class labels y. The purpose of the training is to compute the classifier instance's data structures:
  - `class_priors`, which carry the prior probabilities of each class.
  - `word_counts`, which store the counts of each word in each class.
  - `vocabulary`: the set of unique words ever seen.

  See Sample Run to see what these instance variables should hold after the training is done.

  Note: you should convert each document string to <u>lowercase</u> first before splitting it into words, and then counting their occurrences and putting them into the vocabulary set.

(c) Prediction:

- `predict(self, text: str) -> tuple[str, dict[str, float]]`: this method takes a new document (`text`) as input and predicts its class based on the trained model.

  To carry out the prediction, let's create a dictionary, namely `scores`, to hold the value of $\log P(c) + \sum_{i=1}^{n} \log P(w_i|c)$ for each class label c. Note: $\log$ here means natural logarithm.

  For each class label in `class_priors`, we are going to sum the counts of all words of that class. This gives us $N(c)$ in eq. (4). And for each word in the new `text` to classify, we are to get its word count from `word_counts` using the class label as key. This give us $N(w, c)$ in eq. (4). With the size of the vocabulary $|V|$ known after training, we can apply the Laplace smoothing formula, eq. (4), to get a non-zero word probability, $\log P(w_i|c)$, even for unseen words.

  The method should return the class label with the highest posterior probability, and also the computed `scores` dictionary to allow optional inspection (minor score difference acceptable).

  Note: again, you should convert all words in `text` to lowercase first before the prediction.

Static Method:

(d) Data Loading from File:

- `load_data(filename: str) -> tuple[list[str], list[str]]`: this static method is a helper function that loads training data from a CSV file and returns the list of class labels and the list of text documents in a 2-tuple. The CSV file consists of two columns only: the first column contains the class labels; the second column contains the text documents. For simplicity, no exception handling is required. For your testing, we provided a file called "q2_tweets.csv", which was derived from a dataset, namely Twitter US Airline Sentiment, downloaded from Kaggle and trimmed to 10,000 records with punctuation removed (except single quote).

Sample Client Code:

```
if __name__ == "__main__":
    # Small training dataset
    train_set = [
        ("tech", "I love programming in Python"),
        ("tech", "Python is great for data science"),
        ("outdoors", "I enjoy hiking and outdoor activities"),
        ("tech", "Data science is my passion"),
        ("fashion", "LV and Gucci are top fashion brands"),
    ]
    y_train, X_train = map(list, zip(*train_set))

    # Create and train the classifier
    classifier = NaiveBayesClassifier()
    classifier.fit(X_train, y_train)
    print(f"class_priors:\n{classifier.class_priors}\n")
    print(f"word_counts:\n{classifier.word_counts}\n")
    print(f"vocabulary:\n{classifier.vocabulary}\n")

    # Predict new text
    text_to_predict = "I like Python coding"
    predicted_class, scores = classifier.predict(text_to_predict)
    print(f"scores:\n{scores}")
    print(f"Predicted class for '{text_to_predict}': {predicted_class}")

    # Predict new text
    text_to_predict = "I love outdoor adventures"
    predicted_class, _ = classifier.predict(text_to_predict)
    print(f"Predicted class for '{text_to_predict}': {predicted_class}\n")

    # Load a large training dataset from a CSV file
    y_train, X_train = NaiveBayesClassifier.load_data('q2_tweets.csv')

    # Create and train the classifier
    tweets_classifier = NaiveBayesClassifier()
    tweets_classifier.fit(X_train, y_train)

    # Predict new texts in a test set
    test_set = [
        ("negative", "I am angry at the flight delay"),
        ("negative", "I will complain the staff"),
        ("neutral", "What time will we arrive at Hong Kong"),
        ("positive", "I am happy about the free upgrade"),
        ("positive", "What a happy fun trip with this airline"),
    ]

    print("Batch testing results:")
    correct_predictions = 0
    for true_label, text_to_predict in test_set:
        predicted_class, _ = tweets_classifier.predict(text_to_predict)
        correct = (predicted_class == true_label)
        correct_predictions += correct
        print(predicted_class, correct)
    print(f"Accuracy: {correct_predictions / len(test_set):.2%}")
```

Sample Run:

```
class_priors:
{'tech': 0.6, 'outdoors': 0.2, 'fashion': 0.2}

word_counts:
{'tech': {'i': 1, 'love': 1, 'programming': 1, 'in': 1, 'python': 2, 'is': 2, 'great':
1, 'for': 1, 'data': 2, 'science': 2, 'my': 1, 'passion': 1}, 'outdoors': {'i': 1,
'enjoy': 1, 'hiking': 1, 'and': 1, 'outdoor': 1, 'activities': 1}, 'fashion': {'lv':
1, 'and': 1, 'gucci': 1, 'are': 1, 'top': 1, 'fashion': 1, 'brands': 1}}

vocabulary:
{'python', 'data', 'hiking', 'activities', 'outdoor', 'i', 'passion', 'programming',
'top', 'brands', 'science', 'gucci', 'great', 'enjoy', 'in', 'love', 'my', 'are',
'lv', 'for', 'and', 'fashion', 'is'}

scores:
{'tech':  -13.373312739056523,  'outdoors':  -14.385474051820053,  'fashion':  -
15.214227439082723}
Predicted class for 'I like Python coding': tech
Predicted class for 'I love outdoor adventures': outdoors

Batch testing results:
negative True
negative True
neutral True
negative False
positive True
Accuracy: 80.00%
```

Question 3 (50%)

In this question, you will implement a two-player strategy board game called ***Pentago*** using an OOP approach. This exercise gives you practice with abstract base classes and multiple inheritance.

Introduction of the Pentago Board Game

The Pentago game is played on a 6x6 grid that is divided into four 3x3 sub-boards (called **quadrants**). The two players take turns to place a **marble** (or **stone**) of their color (either black or white) onto an unoccupied cell on the board and then **rotate** one of the quadrants by 90 degrees either clockwise or anticlockwise. The goal of each player is to be the first one to align five adjacent marbles of their color in a line, either horizontally, vertically, or diagonally. Figure 1 shows the game board with an example winning case by placing a marble and then rotating a quadrant. To quickly understand how to play this game, you may watch this YouTube video or play this online game implementing Pentago.

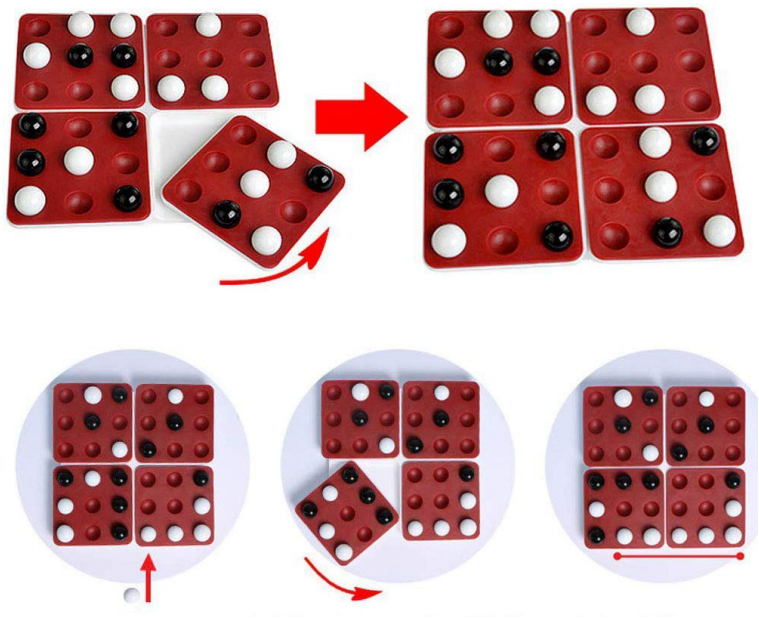


Figure 1: The Pentago board game with an example wining case (image source: thebigbox.co.za)

While the game resembles tic-tac-toe on a larger board, adding quadrant rotation makes the winning strategy much more complex. For ease of discussion, let us define the ordering of the four quadrants as in Figure 2. For example, Quadrant 1 is the left upper sub-board whereas Quadrant 4 is the right bottom sub-board.

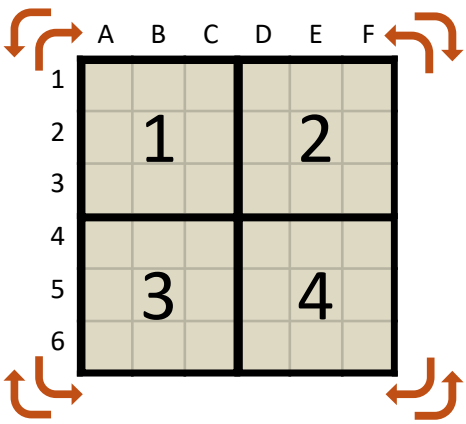


Figure 2: The four quadrants labelled with quadrant ID from 1 to 4.

Below we show some possible game-over scenarios. In Figure 3, the white player can win by placing a white marble at cell C1 and then rotating quadrant 2 clockwise to make five white marbles in a line.

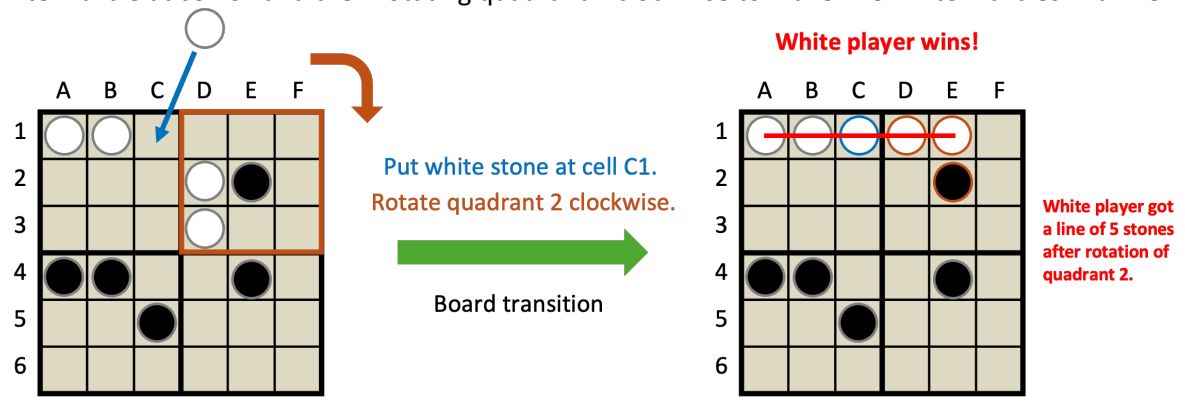


Figure 3: White player wins by placing at cell C1 and rotating quadrant 2.

In another game (Figure 4), the white player attempts to win by placing a marble at cell C1 and rotating quadrant 2. However, the game is a draw because five black marbles also form a line at the same time after the rotation.

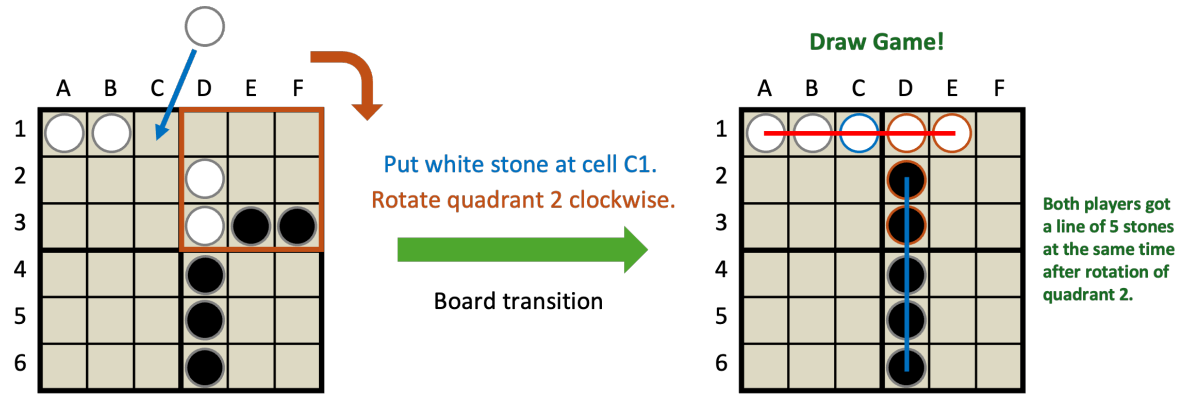


Figure 4: A draw game resulted when the white player places a marble at cell C1 and rotates quadrant 2.

Finally, refer to Figure 5. The black player places a marble at cell B1 and rotates quadrant 2 anticlockwise. This move creates two lines of five black marbles – one horizontal and one diagonal. Although this also concurrently results in one line of five white marbles formed vertically, we consider the player who forms more lines as the winner. Therefore, in this case, the black player wins. We are not aware of any discussion regarding this scenario in the official game rules; therefore, we establish the winning condition as having more lines of five marbles than the opponent.

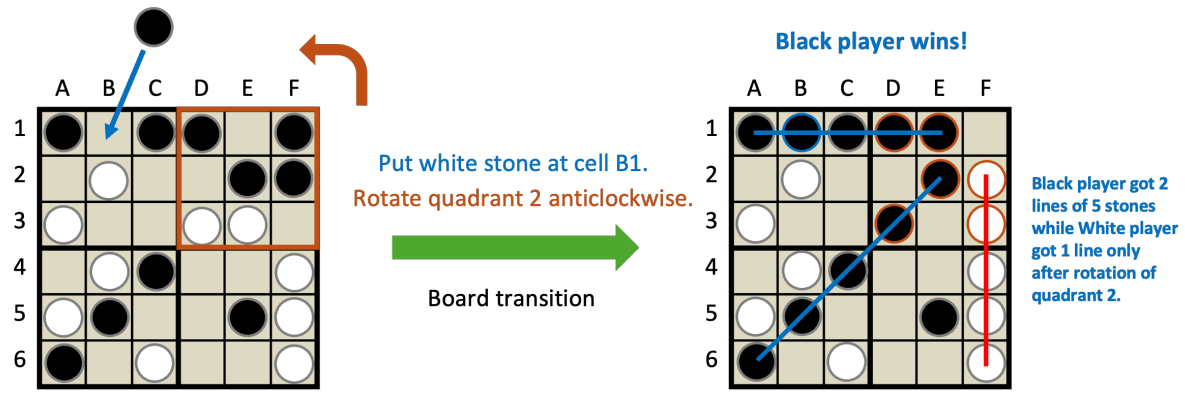


Figure 5: The black wins by placing at B1 and rotating quadrant 2 to form 2 lines while the white forms 1 line only.

Game Board Representation

We are going to implement this game as a console-based program and the game board will be represented by characters arranged in Figure 6 below:

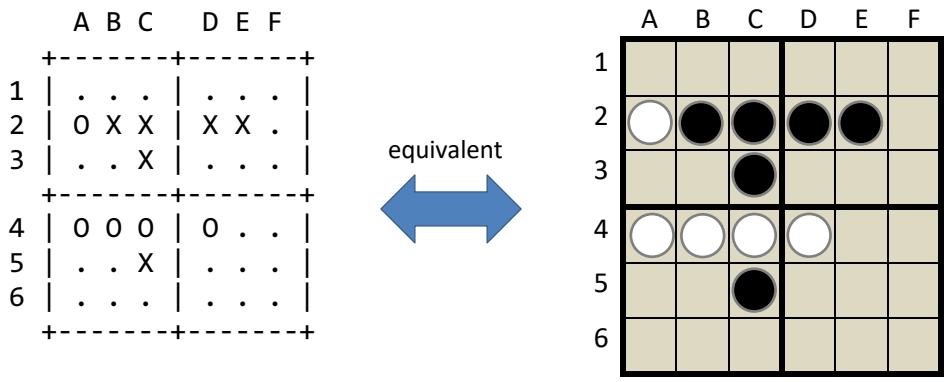


Figure 6: An example game board in the console.

At the start, all board cells are empty (displayed as a dot). The Black and White players' marbles are denoted by the symbols 'X' and 'O', respectively. Each cell is addressed with an Excel-style label; for example, C2 refers to the cell at row 2, column 3. We store the board contents in a 2-D array (implemented as a nested list) of n rows and n columns. This array is an attribute named `cells` in a class called `Matrix`. Each entry in `cells` contains either a `Marble` object (shown as `'X'` or `'O'`) or None (shown as `'.'`). For example, cell C2 is accessed as `cells[1][2]`; in this example, cell C2 holds a reference to a `Marble` object for a black marble (which is printed as `'X'`).

Scalable Board Size and Assumptions

While the original Pentago game assumes a fixed 6×6 board and a line of five consecutive marbles to win, we will enhance its computer version to support a variable board configuration. The board size and the number of marbles required for a winning line should be ***scalable*** (easily configurable) in your code. We may grade your program using different board settings by changing the board size. That said, you may assume the board is always square (n × n), the board size n is always an even integer, and the length of a winning line is always n − 1. There are always four quadrants and only two players.

```
    A B C D   E F G H
  +---------+---------+
1 | . . . . | . . . . |
2 | . . . . | . . . . |
3 | . . . . | . . . . |
4 | . . . . | . . . . |
  +---------+---------+
5 | . . . . | . . . . |
6 | . . . . | . . . . |
7 | . . . . | . . . . |
8 | . . . . | . . . . |
  +---------+---------+
```

(a) An 8x8 game board with 4x4 quadrants

```
    A B C D E   F G H I J
  +-----------+-----------+
1 | . . . . . | . . . . . |
2 | . . . . . | . . . . . |
3 | . . . . . | . . . . . |
4 | . . . . . | . . . . . |
5 | . . . . . | . . . . . |
  +-----------+-----------+
6 | . . . . . | . . . . . |
7 | . . . . . | . . . . . |
8 | . . . . . | . . . . . |
9 | . . . . . | . . . . . |
10 | . . . . . | . . . . . |
  +-----------+-----------+
```

(b) A 10x10 game board with 5x5 quadrants

Figure 7: Game boards of various sizes

The `Pentago` class is the template for creating the board game. It accepts an argument n to define the board size; for example, `game = Pentago(n=6)` creates a game object with a 6×6 game board, which is internally represented by a `Matrix` object whose attributes for the number of rows (`m`) and columns (n) are both set to 6. During testing, you may change n to verify that your implementation still works correctly. For example, if you set n to 8 or 10, your program should print the game boards shown in Figure 7 (a) and (b) respectively. For ease of testing, you may assume that 4 ≤ n ≤ 10. For simplicity, no validation on the parity and range of n is required.

Object-Oriented Design

The OO design for the game is depicted by the following UML class diagram (Figure 8):

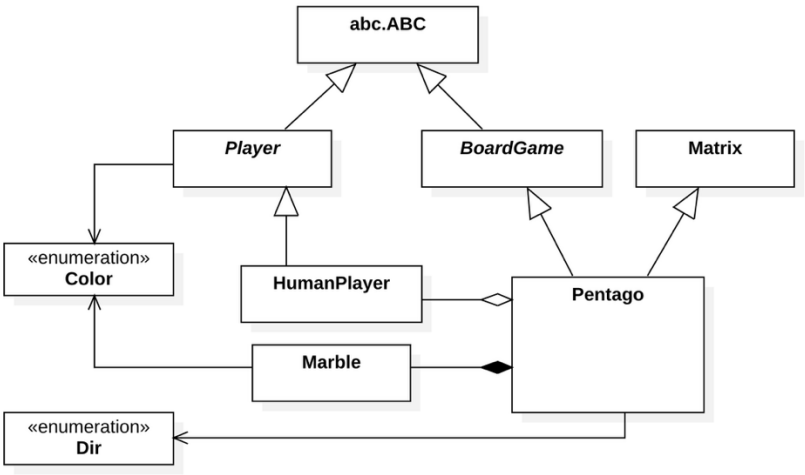


Figure 8: Simplified UML class diagram of the Pentago board game

Below (Figure 9) is a detailed version of the diagram showing each class's the attributes and methods:

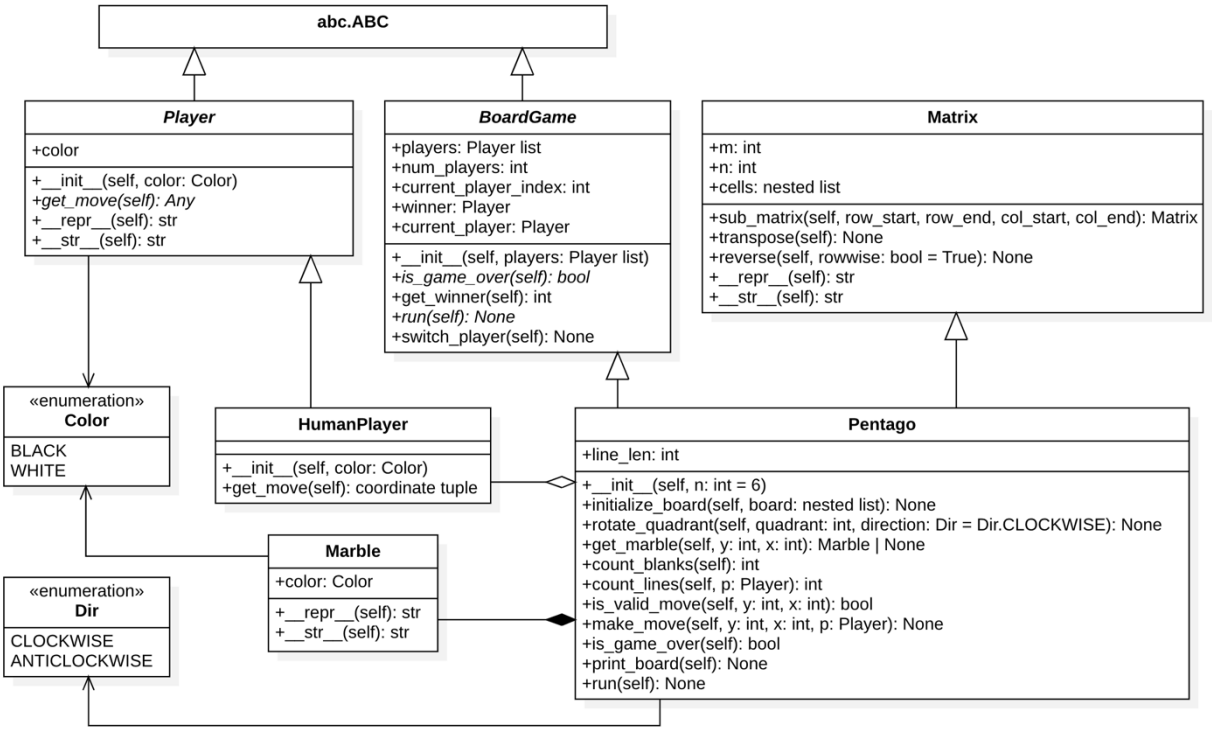


Figure 9: The UML class diagram of the Pentago board game

Notes:

(1) ***Abstract entities***: In the diagrams, class and method names shown in italics indicate that the class or method is *abstract*. The two classes, *`Player`* and *`BoardGame`*, are abstract base classes; their concrete subclasses must override the abstract methods to provide implementations.

(2) ***Enumerations***: The two classes, `Color` and `Dir`, labeled with <<enumeration>> are enumerations. An *enumeration* is a set of symbolic names (members) bound to unique values. In the starter code provided, they are implemented as subclasses of `Enum` imported from the `enum` module.

(3) ***Relationship notations***:
- **Generalization / Inheritance**: solid line with hollow triangle
- **Aggregation**: solid line with an open (white) diamond
- **Composition**: solid line with a filled (black) diamond
- **Directed association**: solid straight line with an open arrowhead between two classes; the arrowhead indicates navigability (which class can access the other)

(4) ***Multiple inheritance***: A strength of this OO design is that the `Pentago` class inherits from both the abstract *`BoardGame`* class and the `Matrix` class, thereby combining and reusing their functionality. This lets `Pentago` reuse matrix operations to simplify some game features. For example, quadrant rotation can be easily implemented by composing the `Matrix` class's transpose and row/column reversal methods.

Implementation

Write a program q3.py that defines several classes specified as follows. Download the starter code file q3_starter.py, rename it, and add your implementation for all TODO tasks in the relevant methods.

Restrictions: To facilitate grading, you must not modify the signature of any of the methods provided. You may add helper (inner or private) functions with names and parameters of your choice if needed to implement those methods. You may also import additional modules from the standard Python library.

Instead of testing your main client code, we will import everything from your q3 module into our test harness. All the methods with TODO tasks may be graded individually via unit tests. Therefore, each method should implement only the behavior described in its specification. Do not put code in a method that goes beyond that method's task description; for example, unless a method's specification instructs otherwise, avoid printing anything from within the method body. Remember to comment out or remove all debugging print statements before submission.

On the other hand, if some functionality is already implemented in a provided method, your code for a TODO task should not duplicate it. Instead, call the appropriate existing method(s) to perform the required work; duplicating functionality may lead to redundant or incorrect implementations.

The `HumanPlayer` Class

The HumanPlayer class represents a human participant in a Pentago game. It inherits from the abstract `Player` base class and manages the logic for capturing moves directly from a person via the console. There is a method with a TODO task for you:

`get_move(self) -> tuple[int, int, Dir, int]`

This method prompts the user for move input, validates it against a required format, and returns the move data as a tuple.

Task and Requirements:

1. **Prompt Message:**
   This method prompts the user to enter a move with a message format: "`Player X's turn: `" or "`Player O's turn: `". Note a trailing space after the colon.
2. **Input Format:**
   The user should enter a single line containing two strings separated by a space (e.g., "B3 C1").
   - **String 1 (Address):** A letter (column) followed by a number (row), such as A1 or F6 (as in Excel).
   - **String 2 (Rotation):** A letter (direction) followed by a number (quadrant), such as C2 (Clockwise, Quadrant 2) or A4 (Anticlockwise, Quadrant 4).
3. **Coordinate Conversion**:
   - Convert the column letter (e.g., A-F) into a 0-based integer index (e.g., 0-5). **Hint**: Use `ord()`.
   - Convert the row number string into a 0-based integer index.
4. **Validation and Error Handling**:
   - The method must be case-insensitive (e.g., "b3 a1" should be treated as "B3 A1").
   - Use the provided `Dir Enum` to validate the rotation direction.
   - The quadrant must be an integer between **1 and 4**.
   - If the input is malformed, has missing parts, or contains invalid values, the method must print an error message "`Invalid format! Enter [Cell][Space][Dir][Quad]`" and prompt the user again until a valid move is entered.
   - **Hint**: Use `try-except` handling.
   - **Note**: Validation that checks if an address refers to an occupied cell or exceeds the game board's boundaries is performed elsewhere, not in this method (which does not know the board size).
5. **Return Value:**
   Return a tuple in the specific format: `(y, x, direction, quadrant)` where:
   - **`y`** (`int`): Row index (0 – n-1).
   - **`x`** (`int`): Column index (0 – n-1).
   - **`direction`** (`Dir`): An instance of the `Dir Enum` (`Dir.CLOCKWISE` or `Dir.ANTICLOCKWISE`).
   - **`quadrant`** (`int`): The chosen quadrant (1–4).

The `Matrix` Class

The `Matrix` class is a utility for managing and manipulating two-dimensional grids. It stores data in a nested list called **`cells`** and tracks dimensions via **`m`** (rows) and **`n`** (columns).

Attributes:

- **`m`** (`int`): The number of rows in the matrix.
- **`n`** (`int`): The number of columns in the matrix.
- **`cells`** (`list[list[Any]]`): The underlying 2D list storing matrix data.

Methods:

Besides those magic methods, this class has provided a `transpose()` method that flips the current matrix over its diagonal, switching rows and columns. There are two methods that require you to finish their TODO tasks as follows.

`sub_matrix(self, row_start: int, row_end: int, col_start: int, col_end: int) -> Self`

This method extracts a specific rectangular portion of the `cells` grid of the current instance into a new `Matrix` instance. It returns a new `Matrix` object representing a sliced portion of the current one. **Note**: your implementation must ensure this function is a ***pure*** one without any side effects.

Example Code:

```
board = Matrix(m=5, n=6, data=[
    [1,  2,  3,  4,  5,  6],
    [7,  8,  9,  10, 11, 12],
    [13, 14, 15, 16, 17, 18],
    [19, 20, 21, 22, 23, 24],
    [25, 26, 27, 28, 29, 30]
])
slice1 = board.sub_matrix(0, 3, 0, 4)
print("slice1:\n", slice1, sep='')
slice2 = board.sub_matrix(2, 5, 3, 6)
print("slice2:\n", slice2, sep='')
slice3 = board.sub_matrix(1, 3, 2, 5)
print("slice3:\n", slice3, sep='')
print(type(slice3))
```

Example Output:

```
slice1:
[[ 1  2  3  4]
 [ 7  8  9 10]
 [13 14 15 16]]
slice2:
[[16 17 18]
 [22 23 24]
 [28 29 30]]
slice3:
[[ 9 10 11]
 [15 16 17]]
<class '__main__.Matrix'>
```

`reverse(self, rowwise: bool = True) -> None`

Reverses the order of elements either row-wise or column-wise. Defaults to row-wise.

Example Code:

```
grid = Matrix(m=3, n=4, data = [
    [1,  2,  3,  4],
    [5,  6,  7,  8],
    [9,  10, 11, 12]
])
print("Original grid:")
print(grid)
print("Reversed elements vertically:")
grid.reverse(True)
print(grid)
print("Further reversed elements horizontally:")
grid.reverse(False)
print(grid)
```

Example Output:

```
Original grid:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

```
Reversed elements vertically:
[[ 9 10 11 12]
 [ 5  6  7  8]
 [ 1  2  3  4]]
Further reversed elements horizontally:
[[12 11 10  9]
 [ 8  7  6  5]
 [ 4  3  2  1]]
```

The Pentago Class

The `Pentago` class acts as the central controller for the Pentago board game. It manages the game state, handles the interaction between players, and enforces the rules of the game.

Given Methods:

`get_marble(self, y: int, x: int) -> Marble | None`

Returns a marble at (y, x) or returns `None` if out of bounds.

`count_blanks(self) -> int`

Returns the total number of empty cells remaining on the board.

`is_valid_move(self, y: int, x: int) -> bool`

This method returns `True` if it is valid to put a marble (regardless of its color) onto the board at row y and column x (both are zero-based indexes), i.e., the array element `cells[y][x]` is storing `None` (representing an empty cell) and not beyond the game board's boundaries, in other words, both y and x must be within [0, n). Otherwise, the function returns `False`.

`make_move(self, y: int, x: int, p: Player) -> None:`

Places a marble of the player's color at the specified coordinates on the board.

`print_board(self) -> None`

Prints the content of the `cells` array to the console in the format as shown in Figure 6 and Figure 7. The textual game board printed includes the column headers (A, B, C, …), row headings (1, 2, 3, …), and edges of the board. You can see more examples in our provided sample runs.

TODO Methods:

`__init__(self, n: int = 6)`

Constructor of the `Pentago` class. It includes the following tasks.

- **Player Initialization**: Create two `HumanPlayer` instances. Assign `Color.BLACK` to the first player and `Color.WHITE` to the second player.
- **Game Setup**: Use the `super()` function to call the `__init__()` method of the immediate parent class (*BoardGame*) by passing a list containing both player objects created.
- **Board Setup**: Use the `super(…)` function to call the `__init__()` method of the next parent class (`Matrix`) to initialize the underlying game board. The board (i.e., the `cells` array in the `Matrix` instance) should be square with dimensions (i.e., m = n).
- **Win Condition**: Define an attribute called `line_len` and set it to n – 1 (e.g., for n = 6, this attribute is set to 5 to represent the 5-in-a-row requirement for the standard 6×6 board).

`rotate_quadrant(self, quadrant: int, direction: Dir = Dir.CLOCKWISE) -> None`

This method rotates the quadrant specified by `quadrant` (possible values: 1, 2, 3, 4) of the board in the direction specified by `direction` ('A' for anticlockwise or 'C' for clockwise). It also prints the message "`Quadrant` *`<quadrant>`* `rotated anticlockwise`" if `direction` equals 'A' and "`Quadrant` *`<quadrant>`* `rotated clockwise`" if `direction` equals 'C'. **Note**: You should use the `Dir` enumeration literals for this comparison instead of using the string literals 'A' and 'C' directly.

Hints: the following hints should be useful for your implementation of this method. The only thing that remains is to use correct start and end positions to slice the matrix (an n/2 × n/2 sub-board) for a specified quadrant ID. Use the appropriate methods in `Matrix` to simplify this method implementation.

90-degree Anticlockwise Rotation

To systematically rotate an $n \times n$ matrix 90 degrees anticlockwise, you can transpose the matrix (swap rows and columns) and then reverse the elements in each column as depicted in Figure 10.

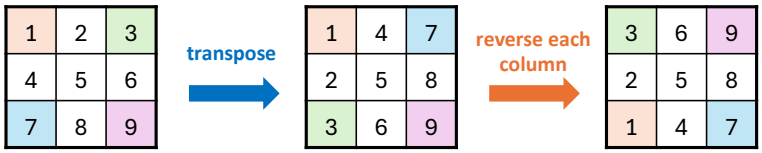


Figure 10: Anticlockwise rotation of a matrix

Step 1: Transpose the matrix.
Step 2: Reverse each column.

90-degree Clockwise Rotation

To systematically rotate an $n \times n$ matrix 90 degrees clockwise, you can transpose the matrix and then reverse the elements in each row as depicted in Figure 11.

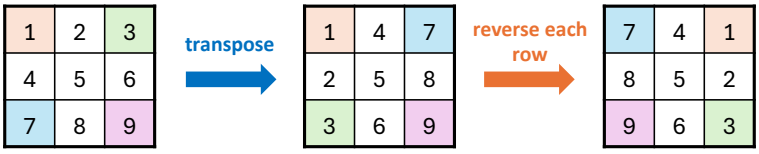


Figure 11: Clockwise rotation of a matrix

Step 1: Transpose the matrix.
Step 2: Reverse each row.

`count_lines(self, p: Player) -> int`

This method returns the number of lines of `line_len` consecutive marbles formed by the player specified by p on the game board. One way to implement this function is to scan the `cells` array in a horizontal, vertical and diagonal access pattern to see if there exist `line_len` adjacent elements (marbles) whose color equals p's color. This method can be used by the `is_game_over()` method to determine whether the game is over.

`is_game_over(self) -> bool`

This method returns `True` if any player has formed <u>at least</u> one line or if the game board is full (i.e., no more empty cells left on the board and hence no more possible moves) and `False` otherwise. It sets the `lines` attribute of both players and the `winner` attribute to the player who forms more lines than the other does. If both players' `lines` are equal, then `winner` remains to be `None` to represent a draw. In this method, you should not print anything about the game-over result. The output indicating who wins and the line counts should be printed by the caller side (the `run` method).

`run(self) -> None`

This method implements the main game loop described as follows. **Note**: You should call the methods specified above to complete the implementation of this method instead of writing redundant code.

1. It starts the game with an empty board. Player X takes the first turn.
2. Print the round information "Round <*round*>:" where <*round*> = 1, 2, 3 … and game board.
3. Then prompt the current player to enter a cell address to place a marble, the quadrant to rotate, and the rotation direction.
4. A marble cannot be placed on the target cell if it is not empty or outside the board's boundaries. If either of these happens, the move is invalid. Display an error message "`Invalid move!`" and go back to Step 3.
5. Perform the validated move by updating the `cells` array by putting a new `Marble` instance of the current player's color at the specified cell location.
6. Check if the game is over after placing the marble.
7. If it is not yet game-over, rotate the specified quadrant in the specified direction.
8. Check again if the game is over after the rotation.
9. If not game-over, switch players to let the opponent take the next turn and increment round.
10. Repeat steps 2–9 until game over, i.e., a player has aligned `line_len` marbles in a row / column / diagonal or the board is full.
11. When the game finishes, print "`Game over:`", the final game board, the count of lines formed by each player in the format "`Player X: <xLines> line(s); Player O: <oLines> line(s)`", where <*xLines*> and <*oLines*> are found and assigned to players' `lines` attributes by the `is_game_over()` method, and the winner status: "`Player X wins!`" or "`Player O wins!`" or "`Draw game!`" accordingly. The winner is the player who has formed more lines than the opponent, e.g., 1 vs. 0, 2 vs. 1, etc. The game draws if <*xLines*> equals <*oLines*>, e.g., 0 vs. 0, and 1 vs. 1.

Sample Client Code:

```
if __name__ == "__main__":
    board_size = int(input("Enter board size [default 6]: ") or 6)
    game = Pentago(n=board_size)
    game.run()
```

Sample Run 1:

The full game log of this sample run is saved in the file `pentago-6x6-run1.txt`.

```
Enter board size [default 6]: 6↵
Round 1:
    A B C   D E F
  +-------+-------+
1 | . . . | . . . |
2 | . . . | . . . |
3 | . . . | . . . |
  +-------+-------+
4 | . . . | . . . |
5 | . . . | . . . |
6 | . . . | . . . |
  +-------+-------+
Player X's turn: abcd↵
Invalid format! Enter [Cell][Space][Dir][Quad]
Player X's turn: 1234 xyz↵
Invalid format! Enter [Cell][Space][Dir][Quad]
Player X's turn: AA BB↵
Invalid format! Enter [Cell][Space][Dir][Quad]
Player X's turn: A3 C8↵
Invalid format! Enter [Cell][Space][Dir][Quad]
```

```
Player X's turn: X3 C1↵
Invalid move!
Player X's turn: D11 A2↵
Invalid move!
Player X's turn: A1 C2↵
Quadrant 2 rotated clockwise
Round 2:
    A B C   D E F
  +-------+-------+
1 | X . . | . . . |
2 | . . . | . . . |
3 | . . . | . . . |
  +-------+-------+
4 | . . . | . . . |
5 | . . . | . . . |
6 | . . . | . . . |
  +-------+-------+
Player O's turn: a3 c2↵
Quadrant 2 rotated clockwise
Round 3:
    A B C   D E F
  +-------+-------+
1 | X . . | . . . |
2 | . . . | . . . |
3 | O . . | . . . |
  +-------+-------+
4 | . . . | . . . |
5 | . . . | . . . |
6 | . . . | . . . |
  +-------+-------+
Player X's turn: C1 C2↵
Quadrant 2 rotated clockwise
Round 4:
    A B C   D E F
  +-------+-------+
1 | X . X | . . . |
2 | . . . | . . . |
3 | O . . | . . . |
  +-------+-------+
4 | . . . | . . . |
5 | . . . | . . . |
6 | . . . | . . . |
  +-------+-------+
Player O's turn: B2 C2↵
Quadrant 2 rotated clockwise
Round 5:
    A B C   D E F
  +-------+-------+
1 | X . X | . . . |
2 | . O . | . . . |
3 | O . . | . . . |
  +-------+-------+
4 | . . . | . . . |
5 | . . . | . . . |
6 | . . . | . . . |
  +-------+-------+
```

...

(Lengthy output truncated due to space constraint)

...

```
Round 21:
    A B C   D E F
  +-------+-------+
1 | X . X | X . X |
2 | . O . | . X X |
3 | O . . | O O . |
  +-------+-------+
4 | . O X | . . O |
5 | O X . | . X O |
6 | X . O | . . O |
  +-------+-------+
Player X's turn: B1 A2↵
Quadrant 2 rotated anticlockwise
Game over:
    A B C   D E F
  +-------+-------+
1 | X X X | X X . |
2 | . O . | . X O |
3 | O . . | X . O |
  +-------+-------+
4 | . O X | . . O |
5 | O X . | . X O |
6 | X . O | . . O |
  +-------+-------+
Player X: 2 line(s); Player O: 1 line(s)
Player X wins!
```

Sample Run 2:

The full game log of this sample run is saved in the file `pentago-6x6-run5.txt`.

```
Enter board size [default 6]: 6↵
Round 1:
    A B C   D E F
  +-------+-------+
1 | . . . | . . . |
2 | . . . | . . . |
3 | . . . | . . . |
  +-------+-------+
4 | . . . | . . . |
5 | . . . | . . . |
6 | . . . | . . . |
  +-------+-------+
Player X's turn: E6 C1↵
Quadrant 1 rotated clockwise
...
(Lengthy output truncated due to space constraint)
...

Round 34:
    A B C   D E F
  +-------+-------+
1 | O X O | X X X |
2 | X X X | O O O |
3 | O O X | X O O |
  +-------+-------+
4 | O X X | O . O |
5 | O X X | X O . |
6 | X X . | O X O |
  +-------+-------+
```

```
Player O's turn: C6 A1↵
Quadrant 1 rotated anticlockwise
Game over:
    A B C   D E F
  +-------+-------+
1 | O X X | X X X |
2 | X X O | O O O |
3 | O X O | X O O |
  +-------+-------+
4 | O X X | O . O |
5 | O X X | X O . |
6 | X X O | O X O |
  +-------+-------+
Player X: 3 line(s); Player O: 0 line(s)
Player X wins!
```

Note: In this sample run, the player X formed a horizontal line of 5 marbles from B1 to F1, a diagonal line of 5 marbles from A2 to E6, and a vertical line of 6 marbles under column B. The program should count this case as three lines for player X. Ensure that your `count_lines()` method implementation correctly counts the number of lines of `line_len` marbles formed. If the number of consecutive marbles in a row, column or diagonal exceeds `line_len`, it should still be counted once only.

Sample Programs and Input Files for More Testing

- **`q3_sample_runs.zip`**: This zip file contains text files storing sample runs (game logs) with different board sizes and sequences of moves.
- **`q3_sample_inputs.zip`**: This zip file contains text files storing the standard input values of moves for each sample run. You can "feed" them directly to your program to automate a long gameplay instead of manually typing each move input in the console to save testing time.
- **`q3_program_macos.zip`**: After unzip, you can find an executable file named `q3sample`, which is a sample program of the Pentago board game we provide for your correctness checks on a macOS platform. You can run it by the command `./q3sample` in the Terminal app.
- **`q3_program_windows.zip`**: After unzip, you can find an executable file named `q3sample.exe`, which is the sample program for a Windows platform. You can run it by the command `.\q3sample` in a Command Prompt or Windows PowerShell.

For more instructions on using these files to streamline the testing of your program implementation, additional guidelines will be provided in a separate help document (`q3-help.pdf`).